ASSIGNMENT NO: 01

AIM:

Study of Deep learning Packages: Tensorflow, Keras, Theano and PyTorch. Document the distinct features and functionality of the packages.

PRE-REQUISITE:

1. Knowledge of Python Programming Language

OBJECTIVE:

1. To understand the concept of the Deep Learning Packages.
2. To understand the concept of Tensorflow, Keras, Theano & PyTorch.

THEORY:

Deep Learning Packages:

Deep learning is a subset of machine learning that uses multilayered neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain. Some form of deep learning powers most of the artificial intelligence (AI) applications in our lives today.
Deep learning is a subfield of machine learning that deals with algorithms inspired by the structure and function of the brain. Deep learning is a subset of machine learning, which is a part of artificial intelligence (AI). Artificial intelligence is the ability of a machine to imitate intelligent human behavior.
A package in deep learning can refer to a model package or a collection of Python packages for machine learning. A package that contains the data and files needed to run deep learning tools for image classification or object detection. It can be uploaded to a portal as a DLPK item and used as input for deep learning raster analysis tools.

A. Keras:

Keras is an effective high-level neural network Application Programming Interface (API) written in Python. This open-source neural network library is designed to provide fast experimentation with deep neural networks, and it can run on top of CNTK, TensorFlow, and Theano. Keras focuses on being modular, user friendly, and extensible. It doesn't handle low-level computations; instead, it hands them off to another library called the Backend.Following are the distinct features of Keras.

- Keras is an API that was made to be easy to learn. It offers consistent & simple APIs, reduces the actions required to implement common code, and explains user error clearly.
- Prototyping time in Keras is less. This means that your ideas can be implemented and deployed in a shorter time. Keras also provides a variety of deployment options depending on user needs.
- Languages with a high level of abstraction and inbuilt features are slow and building custom features can be hard. But Keras runs on top of TensorFlow and is relatively fast. Keras is also deeply integrated with TensorFlow, so you can create customized workflows with ease.
- Keras is used commercially by many companies like Netflix, Uber, Square, Yelp, etc which have deployed products in the public domain which are built using Keras.

Features of Keras:

1. It runs smoothly on both CPU and GPU.
2. It supports almost all neural network models.
3. It is modular in nature, which makes it expressive, flexible, and apt for innovative research.

Steps to build model in Keras:

Step 1:

Define a network: In this step, you define the different layers in our model and the connections between them. Keras has two main types of models: Sequential and Functional models. You choose which type of model you want and then define the dataflow between them.

Step 2:

Compile a network: To compile code means to convert it in a form suitable for the machine to understand. In Keras, the model.compile() method performs this function. To compile the model, we define the loss function which calculates the losses in our model, the optimizer which reduces the loss, and the metrics which is used to find the accuracy of our model.

Step 3:

Fit the network: Using this, we fit our model to our data after compiling. This is used to train the model on our data.

Step 4:

Evaluate the network: After fitting our model, we need to evaluate the error in our model.

Step 5:

Make Predictions: We use model.predict() to make predictions using our

model on new data.

B. PyTorch:

PyTorch is an open-source machine learning Python library that's based on the C programming language framework, Torch. PyTorch qualifies as a data science library and can integrate with other similar Python libraries such as NumPy. It's able to seamlessly create computational graphs that can be changed anytime while the Python program is running. It's mainly used in ML and DL applications such as computer vision and natural language processing.

PyTorch is known for its high speeds of execution even when it's handling heavy and extensive graphs. It's also highly flexible, which allows it to operate on simplified processors in addition to CPUs and GPUs. PyTorch comes with a collection of powerful APIs that lets you expand on the PyTorch library, as well as a natural language toolkit for smoother processing. It's compatible with Python's IDE tools, which makes for an easy debugging process.

PyTorch is an optimized Deep Learning tensor library based on Python and Torch and is mainly used forapplications using GPUs and CPUs. PyTorch is favored over other Deep Learning frameworks like TensorFlow and Keras since it uses dynamic computation graphs and is completely Pythonic. It allowsscientists, developers, and neural network debuggers to run and test portions of the code in real-time. Thus,users don't have to wait for the entire code to be implemented to check if a part of the code works or not.

Features of Keras:

1. Tensor Computation (similar to NumPy) with strong GPU (Graphical Processing) support.
2. Automatic Differentiation for creating and training deep neural networks

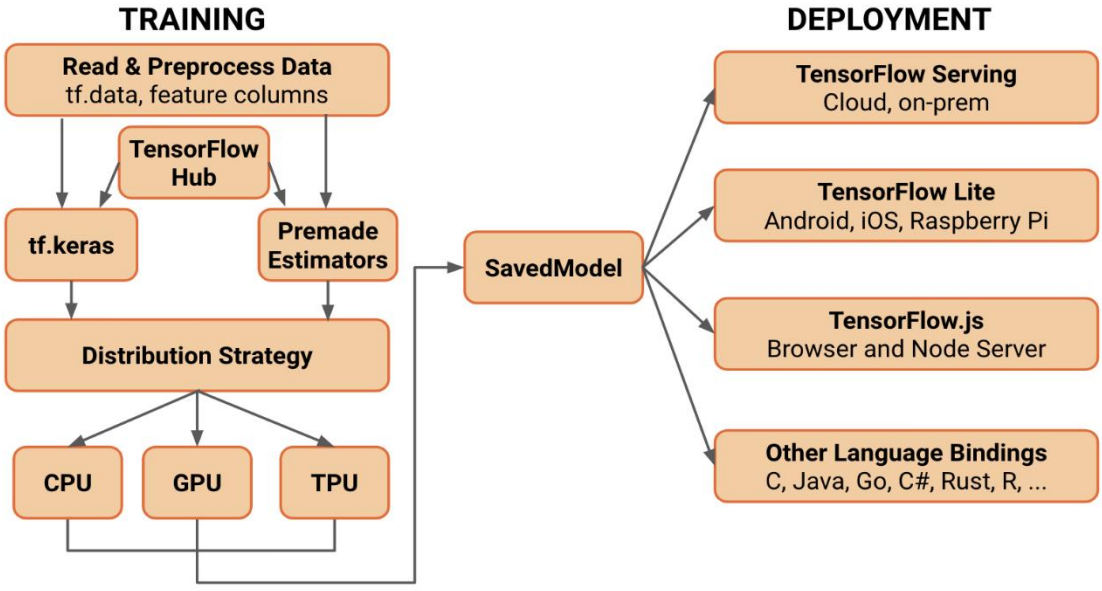
C. TensorFlow:

TensorFlow is an open-source library developed by Google primarily for deep learning applications. It also supports traditional machine learning. TensorFlow was originally developed for large numerical computations without keeping deep learning in mind. However, it proved to be very useful for deep learning development as well, and therefore Google open-sourced it.

TensorFlow accepts data in the form of multi-dimensional arrays of higher dimensions called tensors. Multi-dimensional arrays are very handy in handling large amounts of data. TensorFlow works on the basis of data flow graphs that have nodes and edges. As the execution mechanism is in the form of graphs, it is much easier to execute TensorFlow code in a distributed manner across a cluster of computers while using GPUs. Deep learning applications are very complicated, with the training process requiring a lot of computation. It takes a long time because of the large data size, and it involves several iterative processes, mathematical calculations, matrix multiplications, and so on. If you perform these activities on a normal Central Processing Unit (CPU), typically it would take much longer. Graphical Processing Units (GPUs) are popular in the context of games, where you need the screen and image to be of high resolution.

GPUs were originally designed for this purpose. However, they are being used for developing deep learning applications as well. One of the major advantages of TensorFlow is that it supports GPUs, as well as CPUs. It also has a faster compilation time than other deep learning libraries, like Keras and Torch.TensorFlow allows you to create dataflow graphs that describe how data moves through a graph. The graph consists of nodes that represent a mathematical operation. A connection or edge between nodes is a multidimensional data array. It takes inputs as a multi-dimensional array where you can construct a flowchart of operations that can be performed on these inputs.

TensorFlow Architecture:



Tensorflow architecture works in three significant steps:

- Data pre-processing - structure the data and brings it under one limiting value

- Building the model - build the model for the data
- Training and estimating the model - use the data to train the model and test it with unknown data

Introduction to Components of TensorFlow:

Tensor: Tensor forms the core framework of TensorFlow. All the computations in TensorFlow involve tensors. It is a matrix of n-dimensions that represents multiple types of data. A tensor can be the result of a computation or it can originate from the input data.

Graphs: Graphs describe all the operations that take place during the training. Each operation is called an op node and is connected to the other. The graph shows the op nodes and the connections between the nodes, but it does not display values.

D. Theano:

Theano is a Python library that allows you to define, optimize, and efficiently evaluate mathematical expressions involving multi-dimensional arrays. Theano is a Python library for fast numerical computation that can be run on the CPU or GPU.It is a key foundational library for Deep Learning in Python that you can use directly to create Deep Learning models or wrapper libraries that greatly simplify the process. Theano is a Python library that allows us to evaluate mathematical operations including multi- dimensional arrays so efficiently. It is mostly used in building Deep Learning Projects. It works a way faster on Graphics Processing Unit (GPU) rather than on CPU. Theano attains high speeds that give a tough competition to C implementations for problems involving large amounts of data. It can take advantage of GPUs which makes it perform better than C on a CPU by considerable orders of magnitude under some certain circumstances. It knows how to take structures and convert them into very efficient code that uses NumPy and some native libraries. It is mainly designed to handle the types of computation required for large neural network algorithms used in Deep Learning. That is why, it is a very popular library in the field of Deep Learning

How to install Theano:

Theano can be installed on Windows, MacOS, and Linux. The installation in all the cases is trivial. Before you install Theano, you must install its dependencies. The following is the list of dependencies.

- Python
- NumPy − Required
- SciPy − Required only for Sparse Matrix and special functions
- BLAS − Provides standard building blocks for performing basic vector and matrix operations.

To install Theano and its dependencies, you use **pip** from the command line as follows.

```
$ pip install Theano
$ pip install numpy
$ pip install scipy
$ pip install pydot
```

Advantages of Theano:

1. Stability Optimization: Theano can find out some unstable expressions and can use more stable means to evaluate them
2. Execution Speed Optimization: As mentioned earlier, theano can make use of recent GPUs and execute parts of expressions in your CPU or GPU, making it much faster than Python
3. Symbolic Differentiation: Theano is smart enough to automatically create symbolic graphs for computing gradients. It supports convolutional networks and recurrent networks, as well as combinations of the two

CONCLUSION:

_____

_____

_____

_____

Answer the following questions.

1. What is Packages in Deep Learning?

2. Explain distinct features and functionality of the packages.
3. Explain Tensorflow, Keras deep learning packages in detail.
4. Explain Theano and PyTorch deep learning packages in detail.

ASSIGNMENT NO: 02

AIM:

Implementing Feedforward neural networks with Keras and TensorFlow.

1. Import the necessary packages.
2. Load the training and testing data (MNIST/CIFAR10).
3. Define the network architecture using Keras.
4. Train the model using SGD.
5. Evaluate the network.
6. Plot the training loss and accuracy

PRE-REQUISITE:

1. Knowledge of Python Programming Language

OBJECTIVE:

1. To apply feedforward neural network on MNIST/CIFAR dataset for classification.
2. Evaluate the performance of models and plot the loss and accuracy graph.

THEORY:

Feed Forward Neural Network:

A feedforward neural network is a type of artificial neural network in which nodes' connections do not form a loop.Often referred to as a multi-layered network of neurons, feedforward neural networks are so named because all information flows in a forward manner only.The data enters the input nodes, travels through the hidden layers, and eventually exits the output nodes. The network is devoid of links that would allow the information exiting the output node to be sent back into the network.

A feedforward neural network (FFNN) is a type of artificial neural network that is used in deep learning for tasks like image classification and predictive modeling. It is one of the simplest types of neural networks, and is characterized by the following properties:

1. Data flow:

Information flows in one direction, from input nodes to output nodes, with no loops or cycles.

2. Structure

The network's structure is inspired by the human brain's neuronal system, where signals pass through interconnected neurons.

3. Training

FFNNs are usually trained using supervised learning, where the network is given paired datasets of input and desired output.

4. Performance

The network's performance depends on the weights and biases assigned to its connections.

5. Optimization

Gradient descent is a popular optimization technique for FFNNs.

Structure of a Feedforward Neural Network:

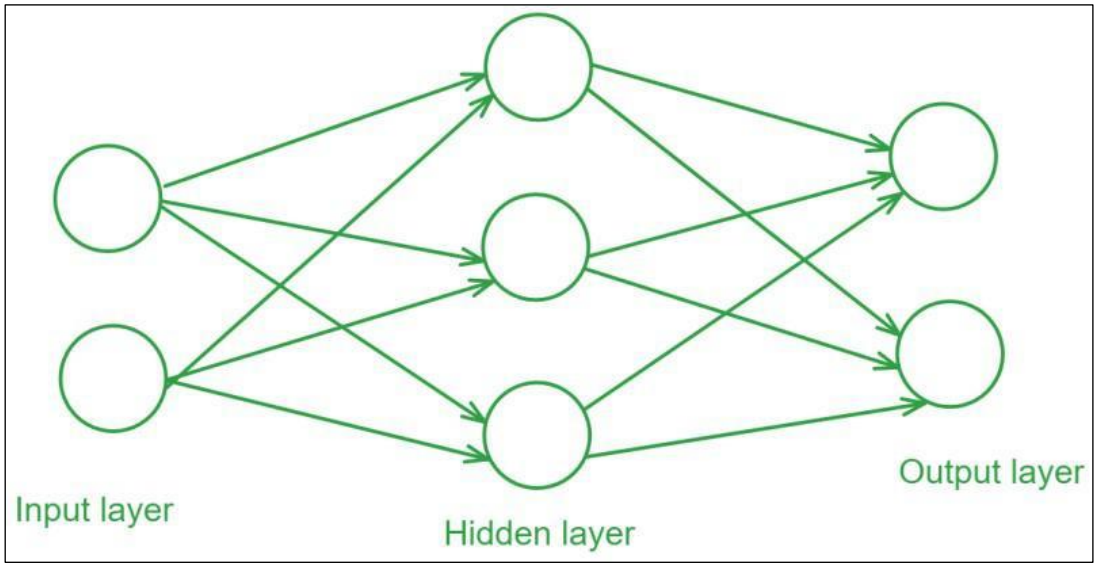


Figure: Feed Forward Neural Network

1. **Input Layer:** The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
2. **Hidden Layers:** One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.

3. **Output Layer:** The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

Neurons weights: Weights are used to describe the strength of a connection between neurons. The range of a weight's value is from 0 to 1.

Cost Function in Feedforward Neural Network:

The cost function is an important factor of a feedforward neural network. Generally, minor adjustments to weights and biases have little effect on the categorized data points. Thus, to determine a method for improving performance by making minor adjustments to weights and biases using a smooth cost function.

The mean square error cost function is defined as follows:

$$C(w, b) \equiv \frac{1}{2n} \sum_{x} \|y(x) - a\|^2.$$

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.

Gradient Learning Algorithm:

Gradient Descent Algorithm repeatedly calculates the next point using gradient at the current location, then scales it (by a learning rate) and subtracts achieved value from the current position (makes a step) (makes a step). It subtracts the value since we want to decrease the function (to increase it would be adding) (to maximize it would be adding).

There's a crucial parameter η which adjusts the gradient and hence affects the step size. In machine learning, it is termed learning rate and has a substantial effect on performance.

- The smaller the learning rate the longer GD converges or may approach maximum iteration before finding the optimal point
- If the learning rate is too great the algorithm may not converge to the ideal point (jump around) or perhaps diverge altogether.

Evaluation of Feedforward neural network

Evaluating the performance of the trained model involves several metrics:

1. **Accuracy**: The proportion of correctly classified instances out of the total instances.

2. **Precision**: The ratio of true positive predictions to the total predicted positives.
3. **Recall**: The ratio of true positive predictions to the actual positives.
4. **F1 Score**: The harmonic mean of precision and recall, providing a balance between the two.
5. **Confusion Matrix:** A table used to describe the performance of a classification model, showing the true positives, true negatives, false positives, and false negatives

CONCLUSION:

_____

_____

_____

_____

_____

Answer the following questions.

1. What is feedforward neural network?
2. How does a Feedforward Neural Network work?
3. What are activation functions?
4. What is the role of the loss function?
5. What is backpropagation?

ASSIGNMENT NO: 03

AIM:

Build the Image classification model by dividing the model into following 4 stages:

1. Loading and preprocessing the image data
2. Defining the model's architecture
3. Training the model
4. Estimating the model's performance

PRE-REQUISITE:

1. Knowledge of Python Programming Language

OBJECTIVE:

1. Apply CNN Deep Learning architecture on MNIST/CIFAR dataset for classification
2. Evaluate the performance of model in terms of accuracy, loss and number of epochs.

THEORY:

What is Image Classification:

Image classification is a computer vision task that involves using deep learning algorithms to assign labels to images.
A computer analyses an image as a matrix of pixels, and then uses algorithms to group the pixels into categories. The algorithm extracts features from the image, such as edges, textures, or shapes, to determine the most appropriate label. Image classification is a supervised learning problem, where a model is trained to recognize a set of target classes using labelled example photos. mage classification is a fundamental project in computer vision that provides valuable data and insights for informed decisions. Deep learning algorithms, such as CNNs, are commonly used for image classification. A well-optimized classification dataset is important for accurate results. Publicly available datasets are often used for training. A trained model's accuracy and precision scores are evaluated on a test set.

Convolutional Neural Network:

A convolutional neural network (CNN) is a type of deep learning algorithm that's used to analyze and learn visual features from data. CNNs are a subset of machine learning and are often used for image recognition and processing.
The goal of a CNN is to learn higher-order features in the data via convolutions. They are well suited to object recognition with images and consistently top image classification competitions. They can identify faces, individuals, street signs,

platypuses, and many other aspects of visual data. CNNs overlap with text analysis via optical character recognition, but they are also useful when analysing words6 as discrete textual units. They're also good at analysing sound. The efficacy of CNNs in image recognition is one of the main reasons why the world recognizes the power of deep learning. As Figure 4-7 illustrates, CNNs are good at building position and (somewhat) rotation invariant features from raw image data. CNNs are powering major advances in machine vision, which has obvious applications for self-driving cars, robotics, drones, and treatments for the visually impaired.

How CNN work:

CNNs use principles from linear algebra, like matrix multiplication, to identify patterns in images. They're made up of multiple layers, including convolution layers, pooling layers, and fully connected layers.

How CNN's are trained:

CNNs require millions of labeled data points for training and high-power processors like a GPU or NPU to produce results quickly.

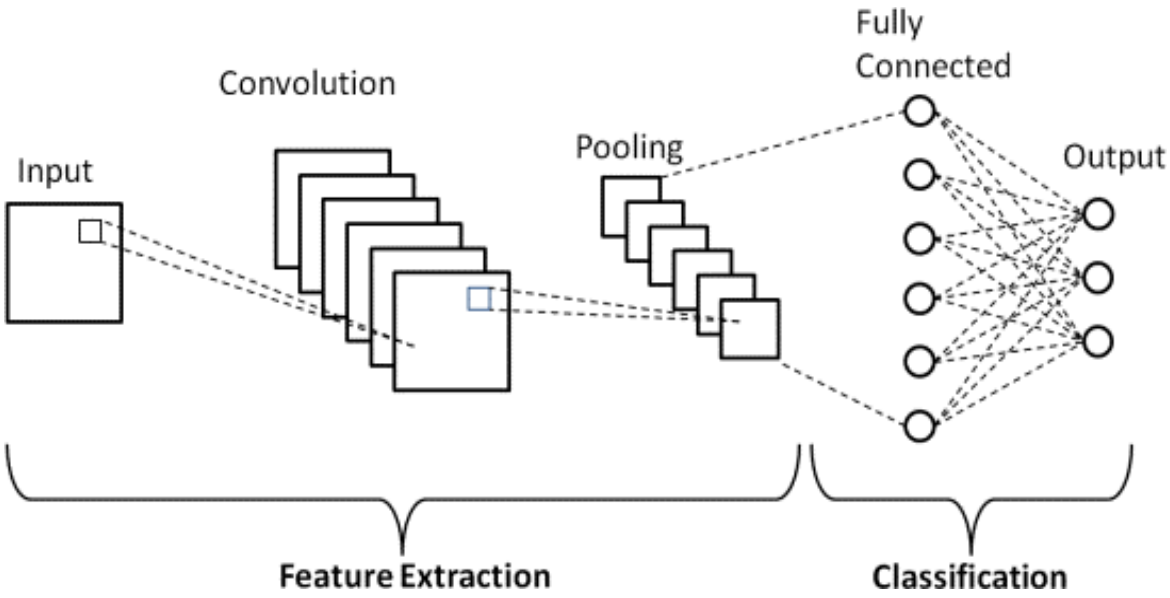
What CNN are used for:

CNNs are primarily used for image-related AI applications, but can also be used for other tasks like natural language processing and recommendation engines.

How CNN are regularized:

CNNs are prone to overfitting data, so regularization techniques like penalizing parameters during training or trimming connectivity can help prevent this.

Architecture of CNN:

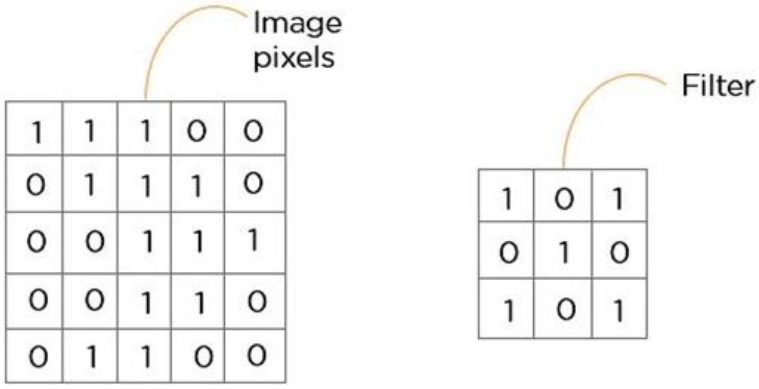


A convolution neural network has multiple hidden layers that help in extracting information from an image. The four important layers in CNN are:

1. Convolution layer
2. ReLU layer
3. Pooling layer
4. Fully connected layer

1. Convolution Layer:

This is the first step in the process of extracting valuable features from an image. A convolution layer has several filters that perform the convolution operation. Every image is considered as a matrix of pixel values. Consider the following 5x5 image whose pixel values are either 0 or 1. There's also a filter matrix with a dimension of 3x3. Slide the filter matrix over the image and compute the dot product to get the convolved feature matrix.

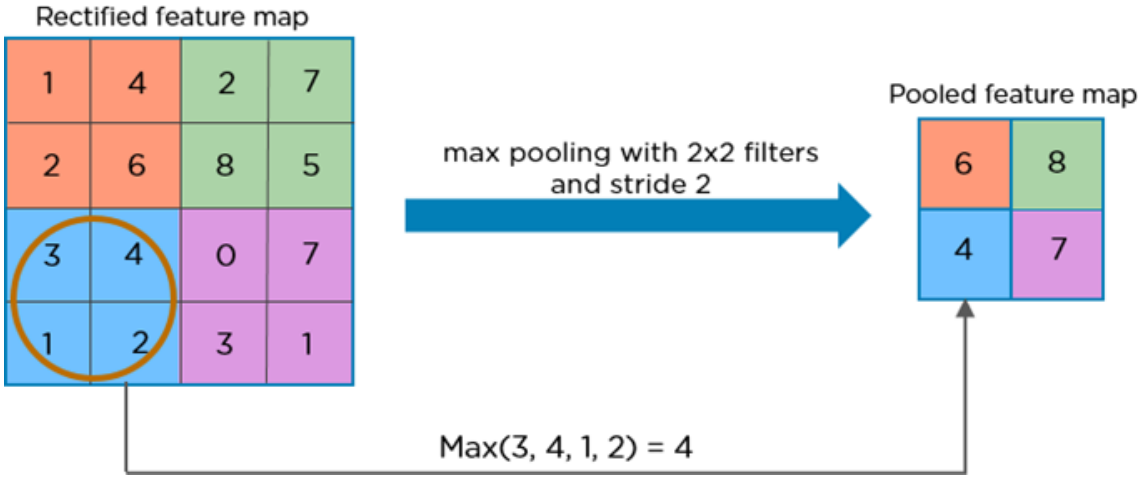


2. ReLU layer:

ReLU stands for the rectified linear unit. Once the feature maps are extracted, the next step is to move them to a ReLU layer. ReLU performs an element-wise operation and sets all the negative pixels to 0. It introduces non-linearity to the network, and the generated output is a rectified feature map. Below is the graph of a ReLU function:

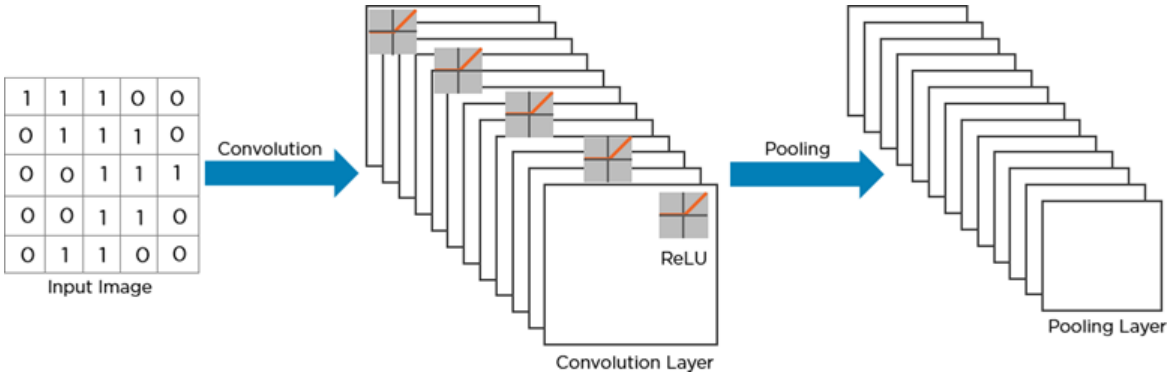
3. Pooling Layer

Pooling is a down-sampling operation that reduces the dimensionality of the feature map. The rectified feature map now goes through a pooling layer to generate a pooled feature map.

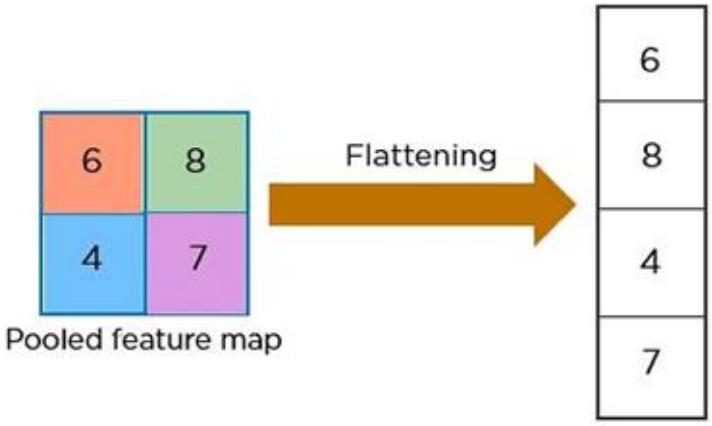


The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes, and beak.

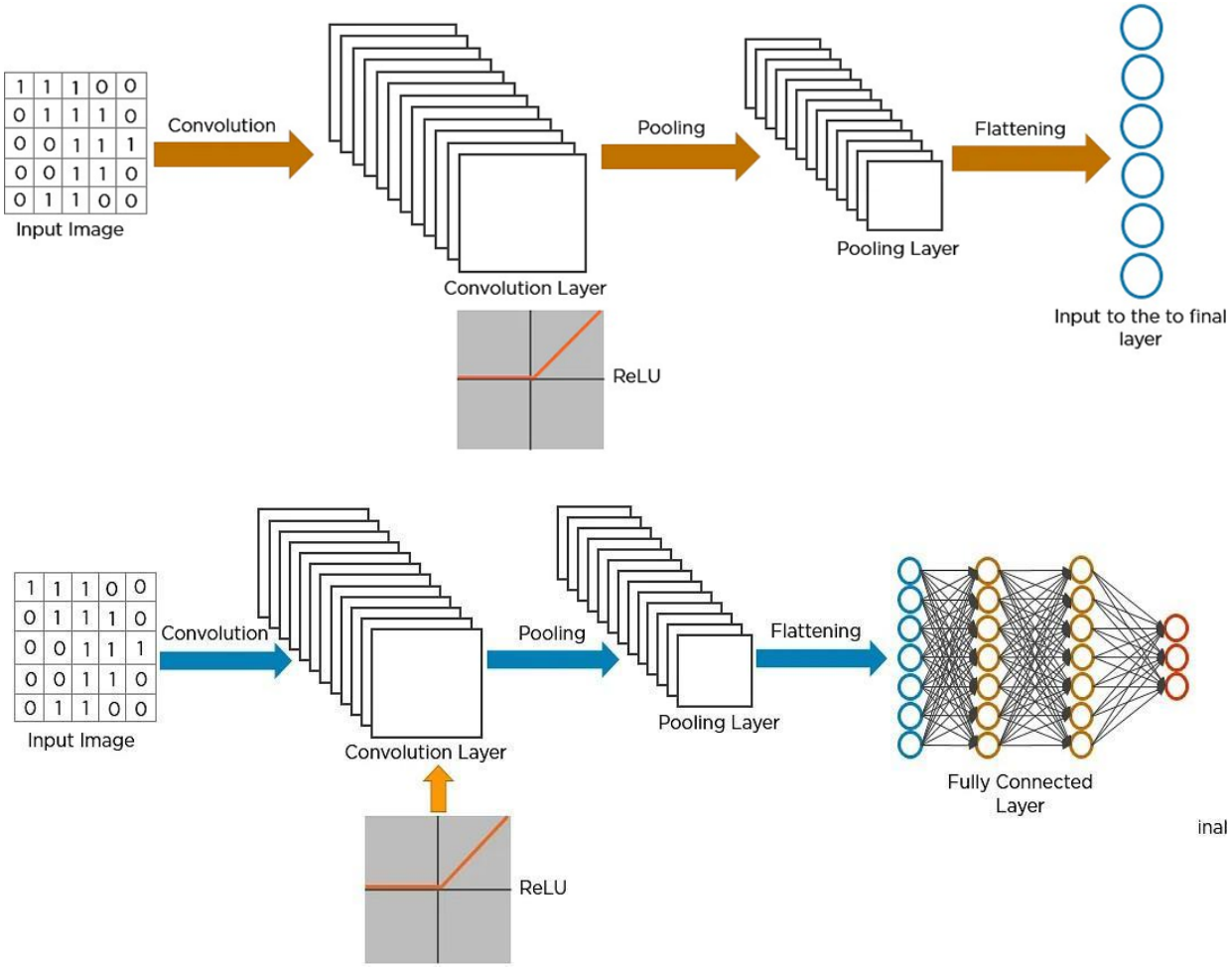
Here's how the structure of the convolution neural network looks so far:



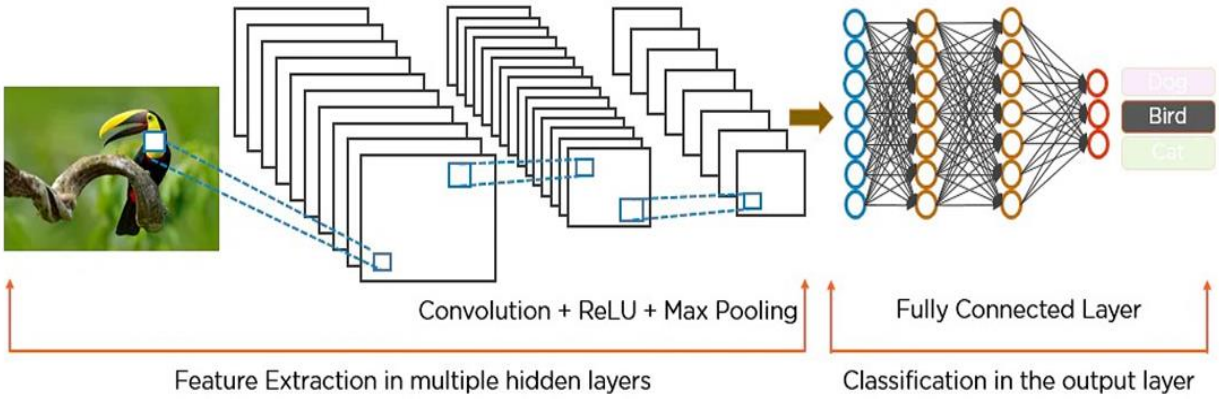
The next step in the process is called flattening. Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector. The flattened matrix is fed as input to the fully connected layer to classify the image.



The flattened matrix is fed as input to the fully connected layer to classify the image.



- The pixels from the image are fed to the convolutional layer that performs the convolution operation
- It results in a convolved map
- The convolved map is applied to a ReLU function to generate a rectified feature map
- The image is processed with multiple convolutions and ReLU layers for locating the features
- Different pooling layers with various filters are used to identify specific parts of the image.
- The pooled feature map is flattened and fed to a fully connected layer to get the final output.



CONCLUSION:

_____

_____

_____

_____

_____

Answer the following questions.

1. What is image classification?
2. How does a Convolutional Neural Network works?
3. What is a Convolutional Neural Network (CNN)
4. What are some common activation functions used in CNNs?
5. What are some common regularization techniques used in CNNs?

ASSIGNMENT NO: 04

AIM:

Use Autoencoder to implement anomaly detection. Build the model by using:

1. Import required libraries
2. Upload / access the dataset
3. Encoder converts it into latent representation
4. Decoder networks convert it back to the original input.
5. Compile the models with Optimizer, Loss, and Evaluation Metrics

PRE-REQUISITE:

1. Knowledge of Python Programming Language

OBJECTIVE:

1. To apply Autoencoder deep learning architecture to determine anomalies in input dataset
2. To Evaluate model.

THEORY:

What is Autoencoders?

In deep learning, autoencoders are a type of neural network that can compress and reconstruct data. An autoencoder is a type of artificial neural network used to learn efficient coding of unlabelled data (unsupervised learning). An autoencoder learns two functions: an encoding function that transforms the input data, and a decoding function that recreates the input data from the encoded representation.

How autoencoders work:

Autoencoders are trained to learn a lower-dimensional representation of higher-dimensional data. They do this by compressing the input data to remove noise and generate a latent space, then reconstructing the original input from the compressed representation

How autoencoders are used:

Autoencoders are useful for unsupervised machine learning tasks such as anomaly detection, data denoising, image inpainting, and information retrieval.

How autoencoders are structured:

Autoencoders are made up of two fully connected feedforward neural networks: an encoder and a decoder. The encoder compresses the data, while the decoder reconstructs it.

How autoencoders are trained:

Autoencoders are trained using unsupervised machine learning to discover latent variables in the input data.

Autoencoders are very useful in the field of unsupervised machine learning. You can use them to compress the data and reduce its dimensionality. The main difference between Autoencoders and Principle Component Analysis (PCA) is that while PCA finds the directions along which you can project the data with maximum variance, Autoencoders reconstruct our original input given just a compressed version of it.If anyone needs the original data can reconstruct it from the compressed data using an Autoencoder. Autoencoders are mainly a dimensionality reduction (or compression) algorithm with a couple of important properties:

- Data-specific: Autoencoders are only able to meaningfully compress data similar to what they have been trained on. Since they learn features specific for the given training data, they are different than a standard data compression algorithm like gzip. So we can't expect an Autoencoder trained on handwritten digits to compress landscape photos.

- Lossy: The output of the Autoencoder will not be exactly the same as the input, it will be a close but degraded representation. If you want lossless compression they are not the way to go.

- Unsupervised: To train an Autoencoder we don't need to do anything fancy, just throw the raw input data at it. Autoencoders are considered an unsupervised learning technique since they don't need explicit labels to train on. But to be more precise they are self-supervised because they generate their own labels from the training data.

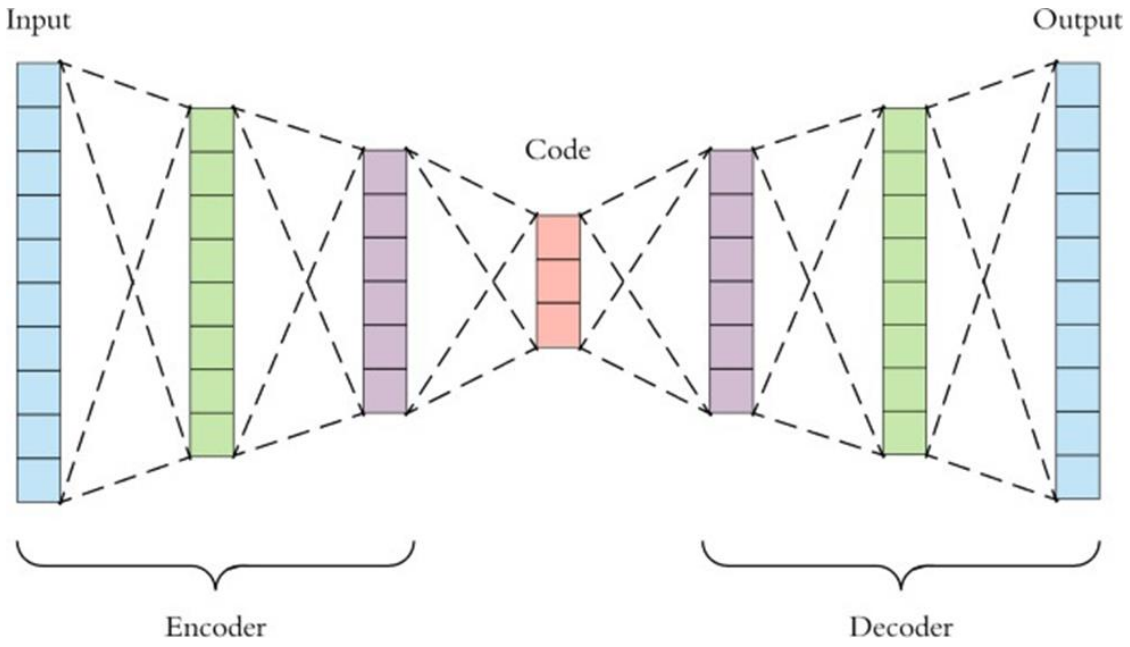
Architecture of CNN:

An Autoencoder is a type of neural network that can learn to reconstruct images, text, and other data from compressed versions of themselves.

An Autoencoder consists of three layers:

1. Encoder
2. Code
3. Decoder

The Encoder layer compresses the input image into a latent space representation. It encodes the input image as a compressed representation in a reduced dimension. The compressed image is a distorted version of the original image.



The Code layer represents the compressed input fed to the decoder layer. The decoder layer decodes the encoded image back to the original dimension. The decoded image is reconstructed from latent space representation, and it is reconstructed from the latent space representation and is a lossy reconstruction of the original image.

There are 4 hyperparameters that we need to set before training an Autoencoder:

- Code size: number of nodes in the middle layer. Smaller size results in more compression.
- Number of layers: the Autoencoder can be as deep as we like. In the figure above we have 2 layers in both the encoder and decoder, without considering the input and output.
- Number of nodes per layer: the Autoencoder architecture is called a stacked Autoencoder since the layers are stacked one after another. The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder. Also the decoder is symmetric to the encoder in terms of layer structure. As noted above this is not necessary and we have total control over these parameters.
- Loss function: we either use mean squared error (MSE) or binary cross entropy. If the input values are in the range [0, 1] then we typically use cross entropy, otherwise we use the mean squared error.

Training of Autoencoder:

Training of an Auto-encoder for data compression: For a data compression procedure, the most important aspect of the compression is the reliability of the reconstruction of the compressed data. This requirement dictates the structure of the Auto-encoder as a bottleneck.

Step 1: Encoding the input data The Auto-encode first tries to encode the data using the initialized weights and biases.

Step 2: Decoding the input data The Auto-encoder tries to reconstruct the original input from the encoded data to test the reliability of the encoding.

Step 3: Backpropagating the error after the reconstruction, the loss function is computed to determine the reliability of the encoding. The error generated is backpropagated.

The above-described training process is reiterated several times until an acceptable level of reconstruction is reached.

After the training process, only the encoder part of the Auto-encoder is retained to encode a similar type of data used in the training process. The different ways to constrain the network are:-

- Keep small Hidden Layers: If the size of each hidden layer is kept as small as possible, then the network will be forced to pick up only the representative features of the data thus encoding the data.

- Regularization: In this method, a loss term is added to the cost function which encourages the network to train in ways other than copying the input.

- Denoising: Another way of constraining the network is to add noise to the input and teach the network how to remove the noise from the data.

- Tuning the Activation Functions: This method involves changing the activation functions of various nodes so that a majority of the nodes are dormant thus effectively reducing the size of the hidden layers.

Variations of Auto-encoderr:

1. Denoising Auto-encoder:

This type of auto-encoder works on a partially corrupted input and trains to recover the original undistorted image. As mentioned above, this method is an effective way to constrain the network from simply copying the input.

2. Sparse Auto-encoder:

This type of auto-encoder typically contains more hidden units than the input but only a few are allowed to be active at once. This property is called the sparsity of the network. The sparsity of the network can be controlled by either manually zeroing the required hidden units, tuning the activation functions or by adding a loss term to the cost function.

3. Variational Auto-encoder:

This type of auto-encoder makes strong assumptions about the distribution of latent variables and uses the Stochastic Gradient Variational Bayes estimator in the training process.

CONCLUSION:

_____

_____

_____

_____

_____

Answer the following questions.

1. What is an autoencoder? Types of Autoencoders.
2. Explain the architecture of a basic autoencoder.
3. What are some key applications of autoencoders?
4. How can autoencoders be used for unsupervised learning?
5. Explain the concept of a sparse autoencoder.

ASSIGNMENT NO: 05

AIM:

Implement the Continuous Bag of Words (CBOW) Model. Stages can be:

1. Data preparation
2. Generate training data
3. Train model
4. Output

PRE-REQUISITE:

1. Knowledge of Python Programming Language

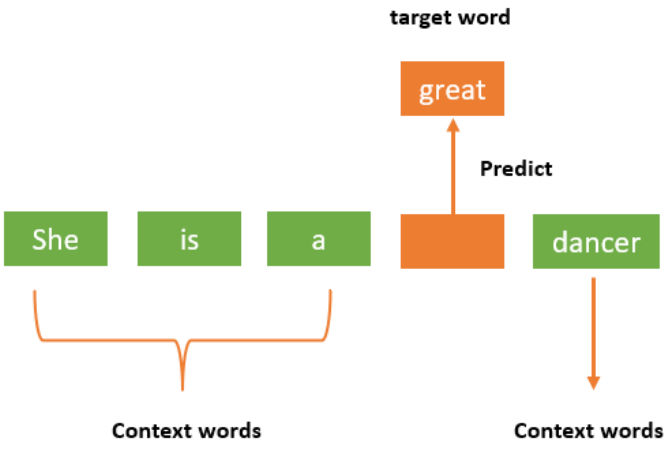
OBJECTIVE:

1. To Build CBOW model to predict the current word given context words within a specific window.
2. To Evaluate model.

THEORY:

What is Continuous bag of words (CBOW)?

Continuous Bag of Words (CBOW) is a popular natural language processing technique used to generate word embeddings. Word embeddings are important for many NLP tasks because they capture semantic and syntactic relationships between words in a language. CBOW is a neural network-based algorithm that predicts a target word given its surrounding context words. It is a type of "unsupervised" learning, meaning that it can learn from unlabelled data, and it is often used to pre-train word embeddings that can be used for various NLP tasks such as sentiment analysis, text classification, and machine translation.



Example of CBOW

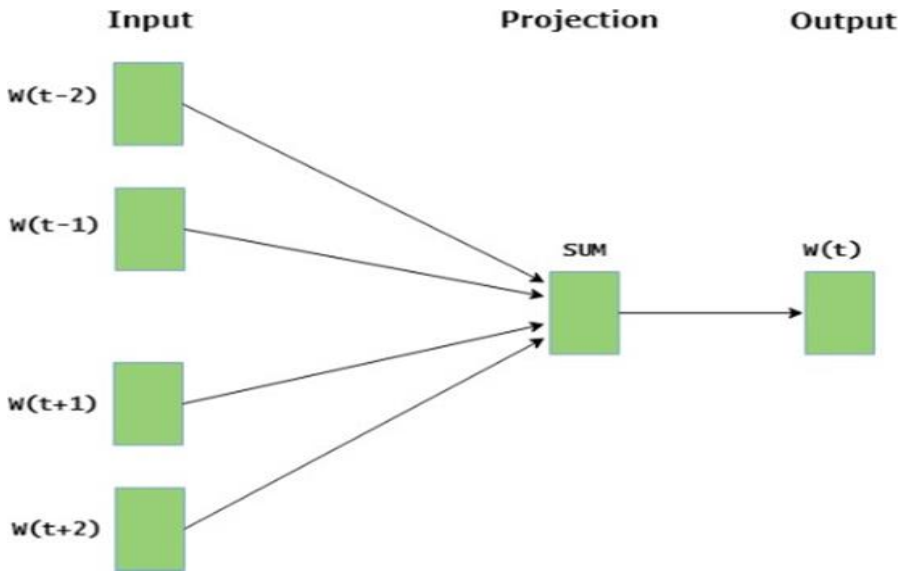
Word Embedding is a language modeling technique used for mapping words to vectors of real numbers. Word embedding can be generated using various methods like neural networks, co-occurrence matrix, probabilistic models, etc. The basic idea of word embedding is words that occur in similar context tend to be closer to each other in vector space.Word2Vec consists of models for generating word embedding. These models are shallow two-layer neural networks having one input layer, one hidden layer, and one output layer. Word2Vec utilizes two architectures:

1. CBOW (Continuous Bag of Words)
2. Skip Gram

1. CBOW (Continuous Bag of Words):

The CBOW model tries to understand the context of the words and takes this as input. It then tries to predict words that are contextually accurate.

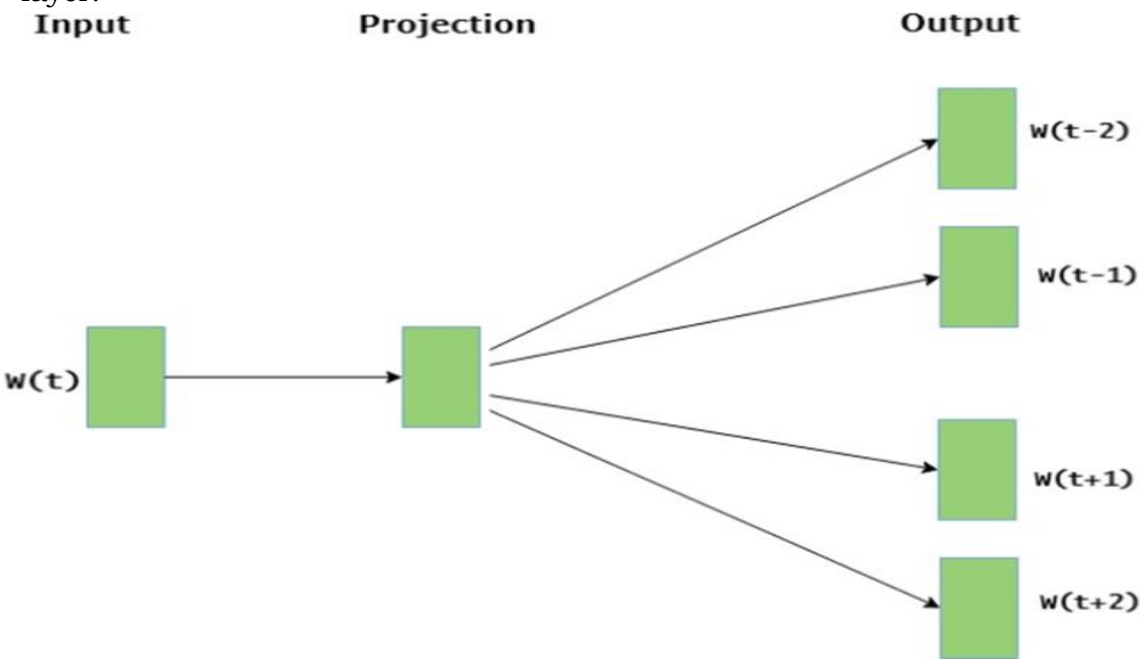
1. CBOW model predicts the current word given context words within a specific window.
2. The input layer contains the context words and the output layer contains the current word.
3. The hidden layer contains the number of dimensions in which we want to represent the current word present at the output layer.



2. Skip Gram :

Skip gram predicts the surrounding context words within specific window given current word. The input layer contains the current word and the output layer contains the context words. The hidden layer contains the number of dimensions in which we want to represent current word present at the input

layer.



The basic idea of word embedding is words that occur in similar context tend to be closer to each other in vector space. For generating word vectors in Python, modules needed are nltk and gensim. Run these commands in terminal to install nltk and gensim:

pip install nltk

pip install gensim

Steps to implement CBOW:
Following are the steps to implement the CBOW Model,

Step 1: Download or collect the dataset from
https://www.gutenberg.org/files/11/11-0.txt

Step 2: Remove all special characters and digits.

Step 3: Remove all punctuation marks Step 4: Bring all letters to lower case.

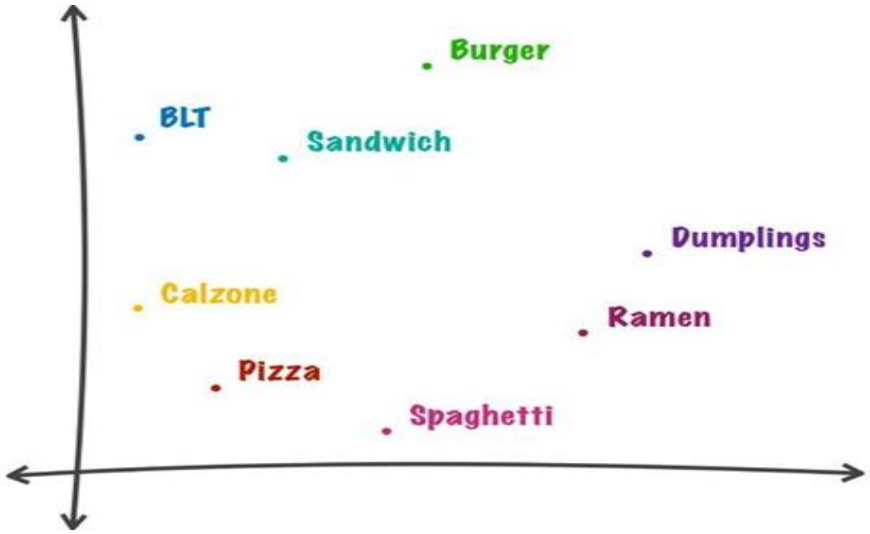
Step 5: Tokenize the document into sentences and words.

Step 6: Remove all stop words.

Step 7: Train the model (Use Gensim word2vec for training the model)

To make words understood by machine learning algorithms, word embedding is used to map words into vectors of real numbers. There are various word

embedding models and word2vec is one of them. In simple words, word2vec is a group of related models that are used to produce word embedding. These models are trained to construct the linguistic contexts of words. Word2vec takes a large corpus of text and produces a vector space, with each unique word in the corpus being assigned to a corresponding vector in the space.



To understand how word2vec works, let's explore some methods that we can use with word2vec such as finding similarities between words or solving analogies like this.

$$f(\text{"canada"}) - f(\text{"us"}) = f(\text{"??"}) - f(\text{"hamburger"})$$

Step 8: Load the dataset and Model.

Step 9: Find the most similar Words.

CONCLUSION:

_____

_____

_____

_____

_____

ASSIGNMENT NO: 06

AIM:

Object detection using Transfer Learning of CNN architectures

1. Load in a pre-trained CNN model trained on a large dataset
2. Freeze parameters (weights) in model's lower convolutional layers
3. Add custom classifier with several layers of trainable parameters to model
4. Train classifier layers on training data available for task.
5. Fine-tune hyper parameters and unfreeze more layers as needed.

PRE-REQUISITE:

1. Knowledge of Python Programming Language

OBJECTIVE:

1. To detect the objects in an image by implementing transfer learning deep learning approach using existing CNN architectures.

THEORY:

What is Transfer Learning?

Whenever we encounter a new problem or a task, we recognize it and apply our relevant knowledge from our previous learning experiences. This makes our work easy and fast to finish. For instance, if you know how to ride a bicycle and if you are asked to ride a motorbike which you have never done before. In such a case, our experience with a bicycle will come into play and handle tasks like balancing the bike, steering, etc. This will make things easier compared to a complete beginner. Following the same approach, a term was introduced Transfer Learning. This approach involves the use of knowledge that was learned in some task and applying it to solve the problem in the related target task.

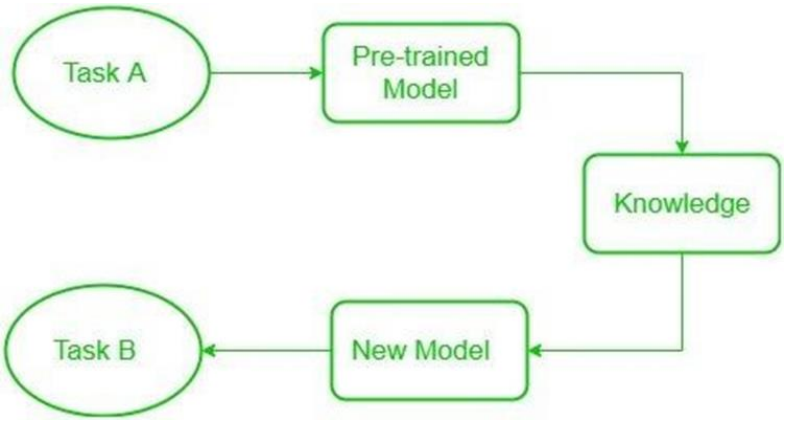
Need of transfer learning:

Many deep neural networks trained on images have a curious phenomenon in common: in the early layers of the network, a deep learning model tries to learn a low level of features, like detecting edges, colors, variations of intensities, etc. Such kind of features appears not to be specific to a particular dataset or a task because no matter what type of image we are processing either for detecting a lion or car. In both cases, we have to detect these low-level features. All these features occur regardless of the exact cost function or image dataset. Thus learning these features in one task of detecting lion can be used in other tasks like detecting humans.

Necessity for transfer learning: Low-level features learned for task A should be beneficial for learning of model for task B. This is what transfer learning is. Nowadays, it is very hard to see people training a whole convolutional neural

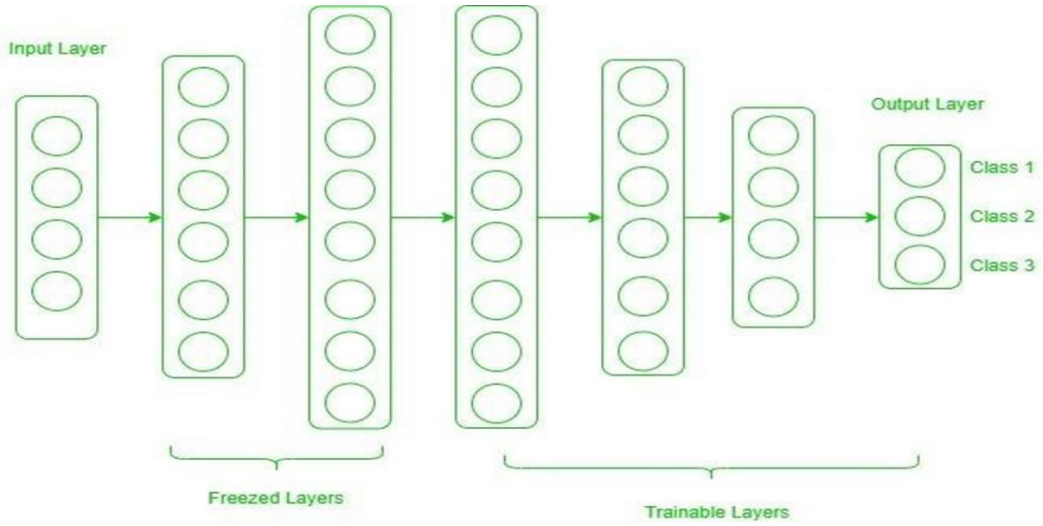
network from scratch, and it is common to use a pre-trained model trained on a variety of images in a similar task, e.g. models trained on ImageNet.

The Block diagram is shown below as follows:



When dealing with transfer learning, we come across a phenomenon called freezing of layers. A layer, it can be a CNN layer, hidden layer, a block of layers, or any subset of a set of all layers, is said to be fixed when it is no longer available to train. Hence, the weights of freezed layers will not be updated during training. While layers that are not freezed follows regular training procedure. When we use transfer learning in solving a problem, we select a pre-trained model as our base model. Now, there are two possible approaches to use knowledge from the pre-trained model. First way is to freeze a few layers of pre-trained model and train other layers on our new dataset for the new task. Second way is to make a new model, but also take out some features from the layers in the pre-trained model and use them in a newly created model. In both cases, we take out some of the learned features and try to train the rest of the model. This makes sure that the only feature that may be same in both of the tasks is taken out from the pre-trained model, and the rest of the model is changed to fit new dataset by training.

Freezed and Trainable Layers:


Input Layer
Output Layer
Class 1
Class 2
Class 3
Freezed Layers
Trainable Layers

CONCLUSION: